

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.* CO (BL5 , BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5 TOTAL MARKS: 50 Annexure A
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I T Sanhith(192311228) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

- 1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.**

This algorithm outlines the cryptographic setup for a secure session, simulating how a client and server agree on encryption parameters and exchange keys.

```
FUNCTION TLS_Handshake(Client, Server):  
    // Phase 1: Client Request  
    client_hello = {  
        "tls_version": "1.3",  
        "cipher_suites": ["AES_256_GCM", "CHACHA20"],  
        "client_random": GENERATE_RANDOM_BYTES(32)  
    }  
    SEND client_hello TO Server  
  
    // Phase 2: Server Authentication & Response  
    server_hello = {  
        "tls_version": "1.3",  
        "selected_cipher": "AES_256_GCM",  
        "server_random": GENERATE_RANDOM_BYTES(32)  
    }  
    server_cert = RETRIEVE_DIGITAL_CERTIFICATE()  
    SEND server_hello, server_cert TO Client  
  
    // Phase 3: Client Verification  
    IF NOT Verify_Certificate_Chain(server_cert):  
        TERMINATE_CONNECTION("Error: Invalid Certificate")  
  
    // Phase 4: Session Key Generation  
    pre_master_secret = GENERATE_RANDOM_BYTES(48)  
    encrypted_secret = ENCRYPT(pre_master_secret, server_cert.public_key)  
    SEND encrypted_secret TO Server  
  
    // Server decrypts the secret
```

```

decrypted_secret = DECRYPT(encrypted_secret, Server.private_key)

// Both parties independently derive the symmetric session key
client_session_key = DERIVE_KEY(client_hello.random, server_hello.random,
pre_master_secret)
server_session_key = DERIVE_KEY(client_hello.random, server_hello.random,
decrypted_secret)

// Phase 5: Secure Data Exchange
SEND ENCRYPT_DATA("Handshake Finished", client_session_key) TO Server
SEND ENCRYPT_DATA("Handshake Finished", server_session_key) TO Client

RETURN "Secure Symmetric Connection Established"

```

2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.

This algorithm processes network traffic sequentially against a predefined Access Control List (ACL), executing a top-down match with an implicit deny at the end.

```

FUNCTION Firewall_Engine(Packet, Rule_Base):
    src_ip = Packet.source_ip
    dst_ip = Packet.destination_ip
    port = Packet.destination_port
    protocol = Packet.protocol

    // Evaluate packet against ordered firewall rules
    FOR EACH rule IN Rule_Base:
        IF (rule.src_ip == "ANY" OR rule.src_ip == src_ip) AND
            (rule.dst_ip == "ANY" OR rule.dst_ip == dst_ip) AND
            (rule.port == "ANY" OR rule.port == port) AND
            (rule.protocol == "ANY" OR rule.protocol == protocol):

            // First match dictates the action
            IF rule.action == "ALLOW":

```

```
    RETURN "FORWARD PACKET"
ELSE IF rule.action == "BLOCK":
    RETURN "DROP PACKET"
```

```
// Implicit Deny (Drop all traffic not explicitly allowed)
RETURN "DROP PACKET"
```

3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction. 4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.

This logic categorizes malicious files based on their delivery mechanisms, host dependencies, and user interaction requirements.

```
FUNCTION Classify_Malware(File_Behavior):
    propagation = File_Behavior.propagation_method
    file_mod = File_Behavior.modifies_host_files
    user_interaction = File_Behavior.requires_user_execution
    deception = File_Behavior.masquerades_as_legitimate

    // Worms spread autonomously over networks without human help
    IF propagation == "Network/Self-Replicating" AND user_interaction == FALSE:
        RETURN "Worm"

    // Viruses attach to legitimate executables and need a user to run them
    ELSE IF propagation == "Infects Host Executables" AND file_mod == TRUE AND
user_interaction == TRUE:
        RETURN "Virus"

    // Trojans pretend to be useful software but hide a malicious payload
    ELSE IF deception == TRUE AND propagation == "None" AND user_interaction ==
TRUE:
        RETURN "Trojan"

    ELSE:
        RETURN "Unknown or Hybrid Malware Variant"
```

4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.

This dual-engine IDS first checks for known bad signatures to catch established threats quickly, then runs statistical anomaly detection to identify zero-day attacks.

```
FUNCTION IDS_Analyze(Network_Traffic, Signature_DB, Baseline_Model):  
  FOR EACH packet IN Network_Traffic:  
  
    match_found = FALSE  
  
    // 1. Signature-Based Detection (Known Threats)  
    FOR EACH signature IN Signature_DB:  
      IF packet.payload CONTAINS signature.byte_pattern:  
        TRIGGER_ALERT(Type="Signature Match", Threat=signature.name)  
        match_found = TRUE  
        BREAK // Stop checking signatures for this packet  
  
    // 2. Anomaly-Based Detection (Unknown/Zero-Day Threats)  
    IF NOT match_found:  
      // Compare current packet behavior against normal network baseline  
      deviation_score = CALCULATE_DEVIATION(packet, Baseline_Model)  
  
      IF deviation_score > ANOMALY_THRESHOLD:  
        TRIGGER_ALERT(Type="Anomaly Detected", Threat="Unusual Traffic Pattern")
```

5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

Unlike an IDS which only alerts, an IPS sits inline and must make real-time decisions to drop malicious packets before they reach the internal network.

```
FUNCTION IPS_Monitor_And_Block(Network_Stream, Threat_Intel):
```

```
WHILE Connection_Is_Active:
```

```
    packet = CAPTURE_PACKET(Network_Stream)
```

```
    // Analyze packet payload and headers
```

```
    threat_level = ANALYZE_PACKET(packet, Threat_Intel)
```

```
    IF threat_level == "MALICIOUS" OR threat_level == "HIGH_RISK":
```

```
        // Action: Block
```

```
        BLOCK_SOURCE_IP(packet.source_ip)
```

```
        // Action: Log
```

```
        LOG_EVENT(
```

```
            Timestamp = CURRENT_TIME(),
```

```
            Source = packet.source_ip,
```

```
            Action = "DROPPED",
```

```
            Reason = "Malicious payload detected"
```

```
        )
```

```
    ELSE:
```

```
        // Action: Allow
```

```
        FORWARD_PACKET(packet)
```

```
        // Action: Log
```

```
        LOG_EVENT(
```

```
            Timestamp = CURRENT_TIME(),
```

```
            Source = packet.source_ip,
```

```
            Action = "ALLOWED"
```

```
        )
```